Experiment 9

ERC-20 Token Creation and Deployment using Remix IDE and MetaMask

Aim

To develop an ERC-20 token using Solidity in Remix IDE and deploy its smart contract on the Sepolia test network using MetaMask. To understand the working of token attributes, functions and events through practical implementation.

Theory

ERC-20 is a commonly used technical standard for creating fungible tokens on the Ethereum blockchain and other compatible networks. Fungible tokens are digital assets in which every unit has the same value and characteristics as another unit of that token. For example, one unit of a particular ERC-20 token can be exchanged for another unit of the same token.

An ERC-20 token is implemented through a smart contract written in Solidity. The contract maintains the token balances of different wallet addresses and provides predefined functions through which users can transfer tokens, approve other accounts to spend tokens and check available balances.

The important components of the ERC-20 standard are:

Member	Type	Description
name	Attribute	Specifies the complete name of the token.
symbol	Attribute	Represents the token using a short identifying code.
decimals	Attribute	Defines the decimal precision of the token. A value of 18 is commonly used.
totalSupply	Attribute	Stores the overall quantity of tokens created.

balanceOf(owner)	Function	Displays the number of tokens held by a particular wallet.
transfer(to, amount)	Function	Allows a wallet owner to transfer tokens to another address.
approve(spender, amount)	Function	Gives another address permission to spend a specified number of tokens.
allowance(owner, spender)	Function	Checks the remaining spending permission given to another address.
transferFrom(from, to, amount)	Function	Enables an approved account to transfer tokens on behalf of the original owner.
Transfer	Event	Records token movements, including the initial creation of tokens.
Approval	Event	Records the permissions granted by one account to another.

Solidity is used to develop the smart contract, while Remix IDE provides an online platform for writing, compiling and deploying Solidity programs. MetaMask works as a cryptocurrency wallet and connects the browser to the Ethereum network. It is also responsible for requesting user approval and signing transactions before they are executed.

In this experiment, the Sepolia test network is used instead of the Ethereum mainnet. Sepolia provides a testing environment where developers can deploy and interact with smart contracts using test ETH, which has no real monetary value. Transactions that modify blockchain data, such as deployment and token transfers, require gas fees. However, operations that only read contract information, such as checking the token name or balance, do not require transaction fees.

The smart contract also contains conditions that prevent invalid operations. For example, a user cannot transfer more tokens than the balance available in their wallet. Such an operation is rejected by the contract, and the transaction is reverted.

Steps

1. Open the Remix IDE at remix.ethereum.org, create a file named MyToken.sol and write the ERC-20 contract: the name, symbol and decimals attributes, totalSupply, the balanceOf and allowance mappings, the Transfer and Approval events, and the transfer,

approve and transferFrom functions. The constructor mints the initial supply to the deployer.

2. Compile the contract with Solidity 0.8.20 or newer and confirm that Remix shows it as Compiled (Fig. 1).
3. In Deploy & Run Transactions, set the environment to Browser Extension (MetaMask) with the Sepolia test network selected, and choose the MetaMask account that will deploy the token.
4. Enter 1000000 as initialSupply, click Deploy, and confirm the "Deploy a contract" request in MetaMask (Fig. 2).
5. Check the deployment transaction and the contract verification output in the Remix console (Fig. 3).
6. Under Deployed Contracts, call the read functions name, symbol and totalSupply to check the token attributes (Fig. 4).
7. Call transfer to send 1 token (1000000000000000000 units) to a second MetaMask account and confirm it in MetaMask (Fig. 5). Open the transaction in the console to see its status and logged event (Fig. 6).
8. As a negative test, try a transfer from an account that holds no tokens and note the warning MetaMask shows (Fig. 7).

Output

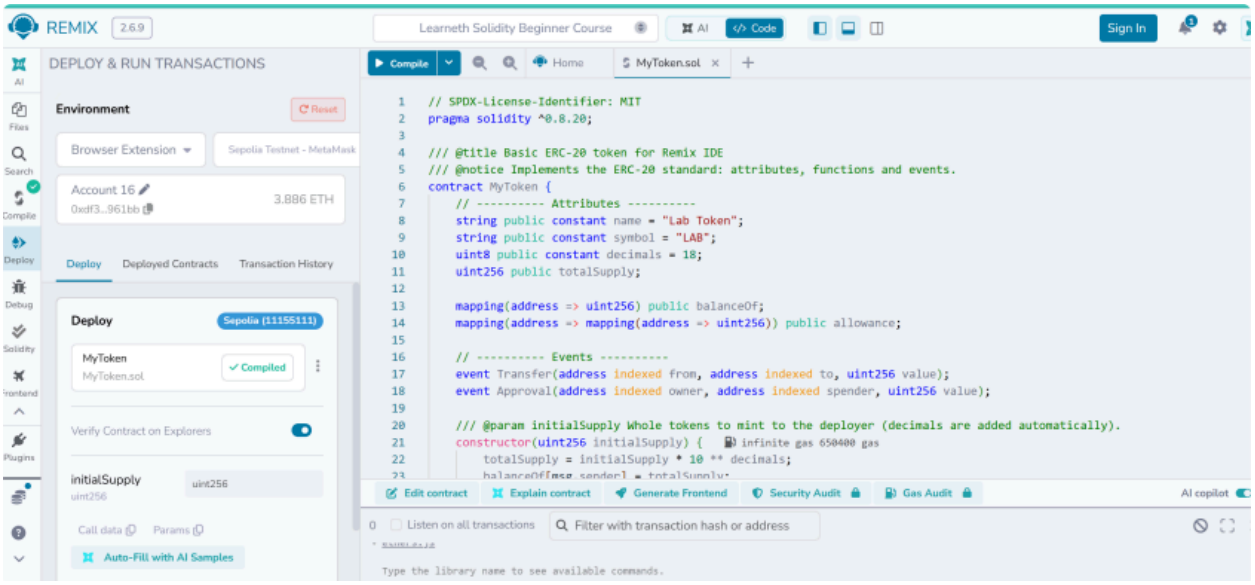


Fig. 1: The MyToken.sol smart contract in Remix IDE after successful compilation, with the Sepolia test network and the deploying MetaMask account selected.

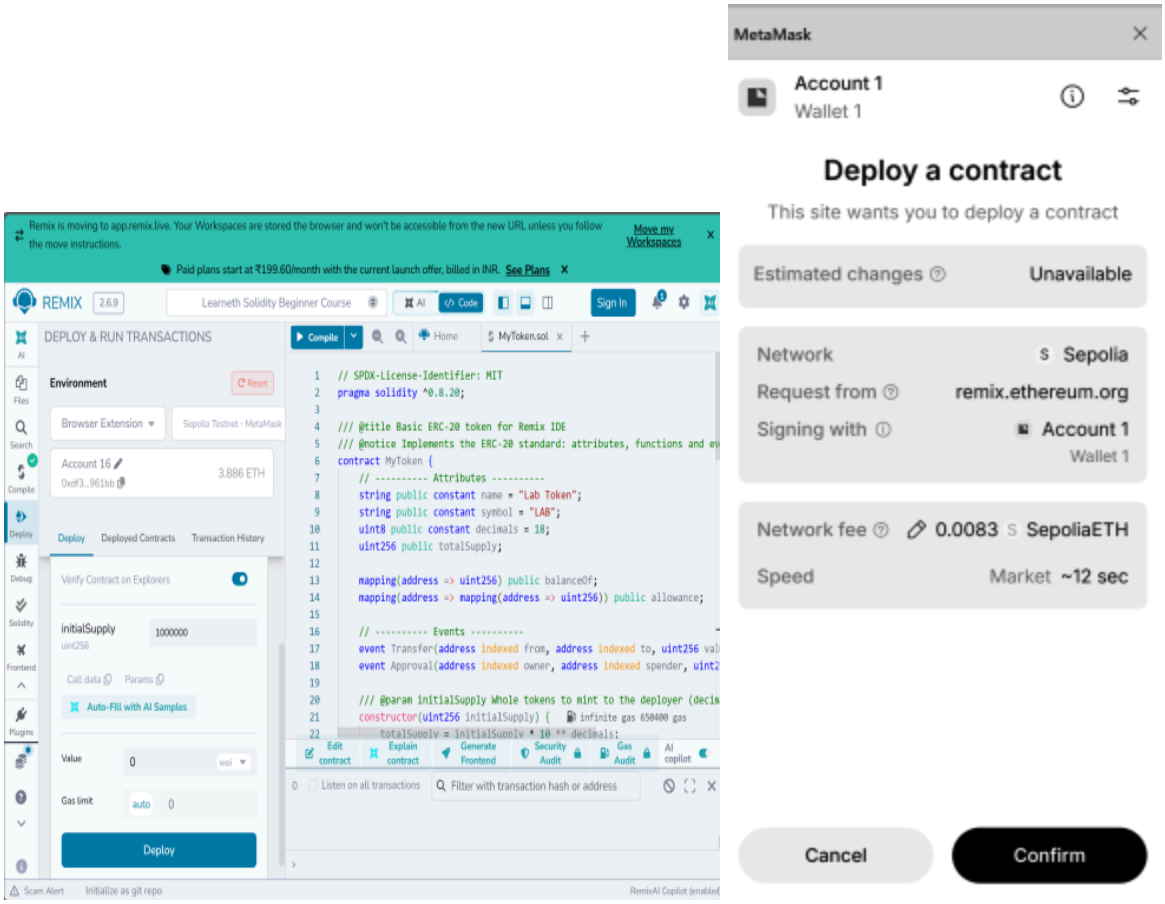


Fig. 2: Deployment of the ERC-20 token with the initial supply set to 1,000,000 tokens, followed by the MetaMask confirmation request.



Fig. 3: Successful execution of the deployment transaction and the contract verification status displayed in the Remix console.

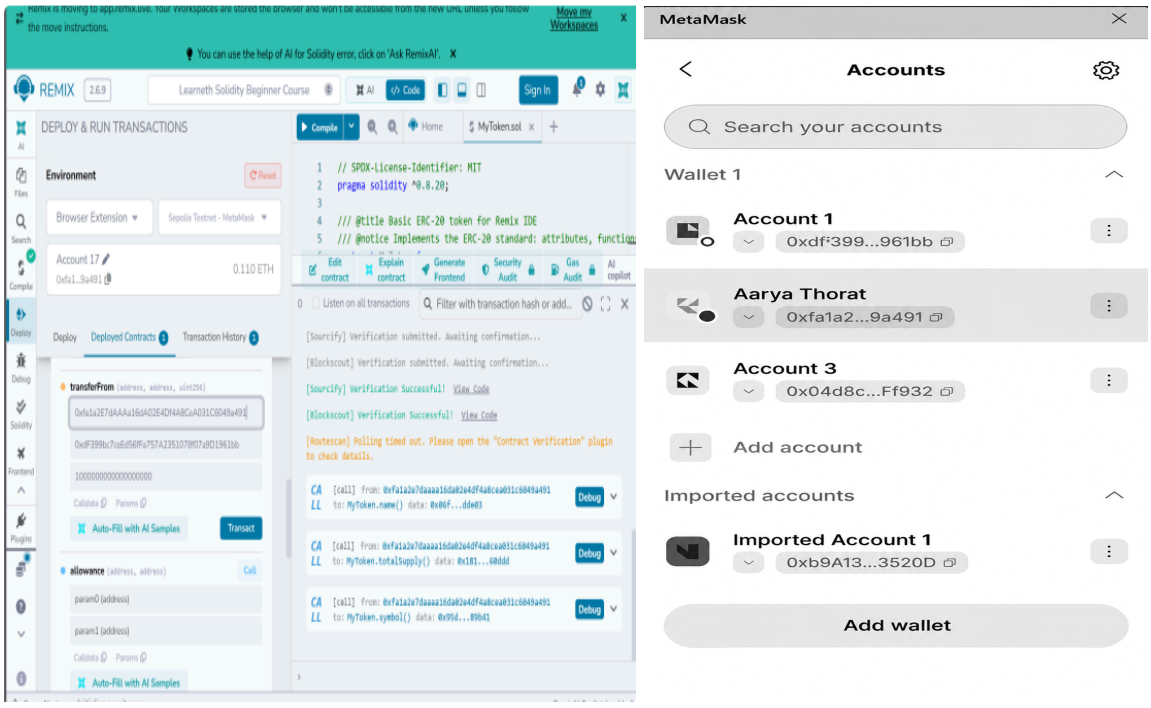


Fig. 4: The deployed MyToken contract showing the results of the name(), symbol() and totalSupply() functions, along with the MetaMask accounts used for testing.

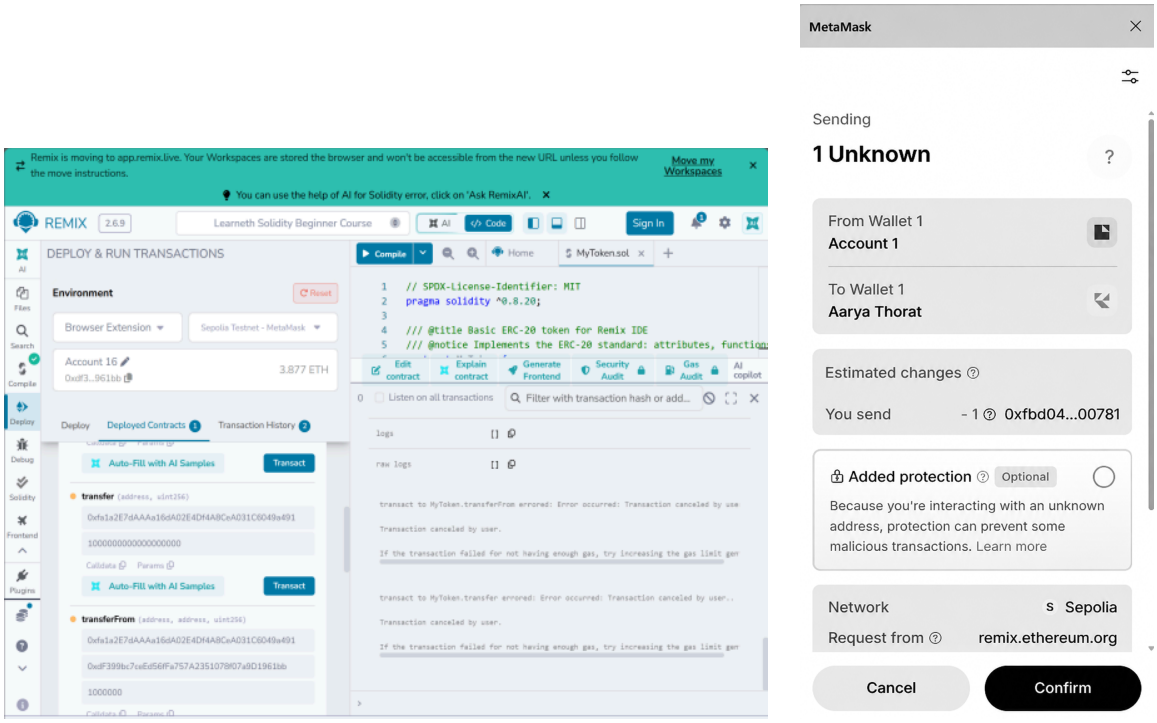


Fig. 5: MetaMask transaction confirmation screen displaying the transfer of 1 token from the first account to the second account (Aarya Thorat) on the Sepolia network.

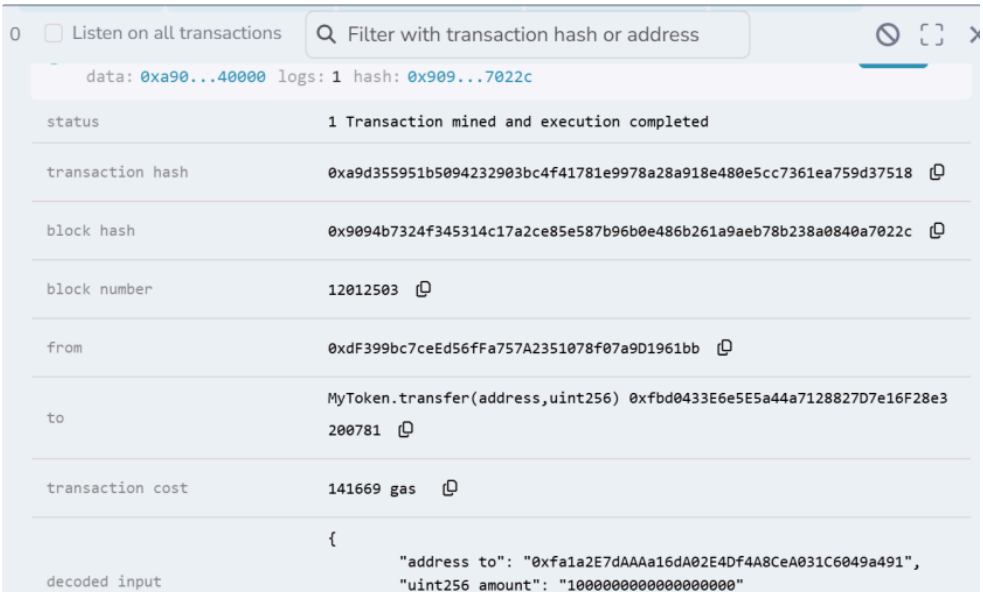


Fig. 6: Transaction details showing the successful mining of the token transfer, including the decoded input amount and the Transfer event generated by the contract.

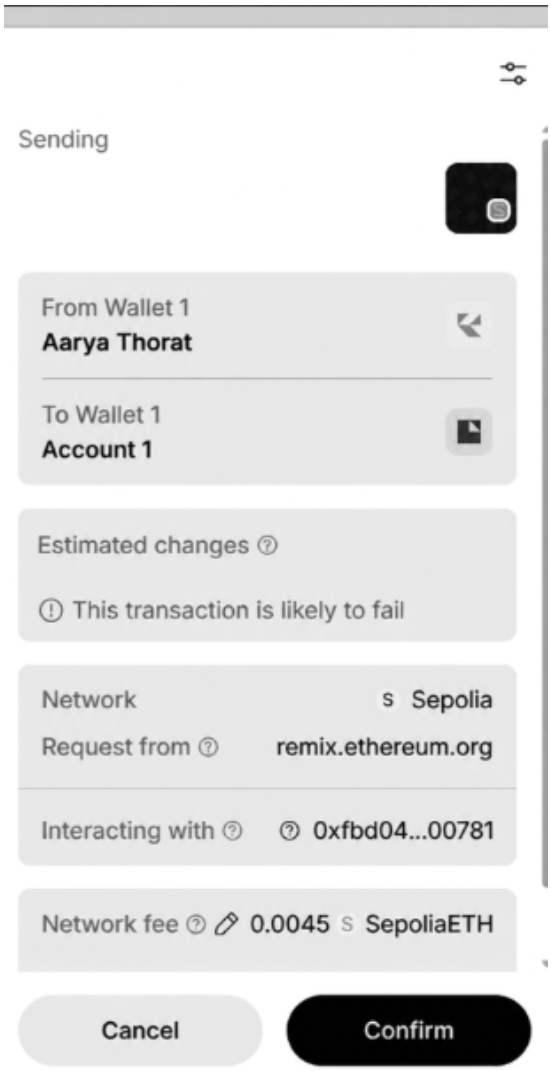


Fig. 7: MetaMask displaying a transaction failure warning when an account without sufficient token balance attempts to transfer tokens.

Conclusion

In this experiment, an ERC-20 token named Lab Token (LAB) was created using Solidity and deployed successfully on the Sepolia test network through Remix IDE and MetaMask. The token was configured with 18 decimal places and an initial supply of 1,000,000 tokens.

The deployed contract was tested by retrieving its name, symbol and total supply. A transfer of one token between two MetaMask accounts was also performed, demonstrating how the transfer function and Transfer event operate. Additionally, an invalid transfer was attempted from an account without sufficient tokens, resulting in a transaction failure warning.

Thus, the experiment provided practical understanding of ERC-20 token development, smart contract deployment, wallet integration and token transactions on an Ethereum-compatible blockchain.